

NAME

ioctl_pagemap_scan – get and/or clear page flags

LIBRARY

Standard C library (*libc*, *-lc*)

SYNOPSIS

```
#include <linux/fs.h> /* Definition of struct pm_scan_arg,
                        struct page_region, and PAGE_IS_* constants */
#include <sys/ioctl.h>

int ioctl(int pagemap_fd, PAGEMAP_SCAN, struct pm_scan_arg *arg);
```

DESCRIPTION

This **ioctl(2)** is used to get and optionally clear some specific flags from page table entries. The information is returned with **PAGE_SIZE** granularity.

To start tracking the written state (flag) of a page or range of memory, the **UFFD_FEATURE_WP_ASYNC** must be enabled by **UFFDIO_API ioctl(2)** on **userfaultfd** and memory range must be registered with **UFFDIO_REGISTER ioctl(2)** in **UFFDIO_REGISTER_MODE_WP** mode.

Supported page flags

The following page table entry flags are supported:

PAGE_IS_WPALLOWED

The page has asynchronous write-protection enabled.

PAGE_IS_WRITTEN

The page has been written to from the time it was write protected.

PAGE_IS_FILE

The page is file backed.

PAGE_IS_PRESENT

The page is present in the memory.

PAGE_IS_SWAPPED

The page is swapped.

PAGE_IS_PFNZERO

The page has zero PFN.

PAGE_IS_HUGE

The page is THP or Hugetlb backed.

Supported operations

The get operation is always performed if the output buffer is specified. The other operations are as following:

PM_SCAN_WP_MATCHING

Write protect the matched pages.

PM_SCAN_CHECK_WPASYNC

Abort the scan when a page is found which doesn't have the Userfaultfd Asynchronous Write protection enabled.

The struct pm_scan_arg argument

```
struct pm_scan_arg {
    __u64  size;
    __u64  flags;
    __u64  start;
    __u64  end;
    __u64  walk_end;
    __u64  vec;
    __u64  vec_len;
```

```

    __u64  max_pages;
    __u64  category_inverted;
    __u64  category_mask;
    __u64  category_anyof_mask;
    __u64  return_mask;
};

```

size This field should be set to the size of the structure in bytes, as in `sizeof(struct pm_scan_arg)`.

flags The operations to be performed are specified in it.

start The starting address of the scan is specified in it.

end The ending address of the scan is specified in it.

walk_end

The kernel returns the scan's ending address in it. The `walk_end` equal to `end` means that scan has completed on the entire range.

vec The address of `page_region` array for output.

```

    struct page_region {
        __u64  start;
        __u64  end;
        __u64  categories;
    };

```

vec_len

The length of the `page_region` struct array.

max_pages

It is the optional limit for the number of output pages required.

category_inverted

PAGE_IS_* categories which values match if 0 instead of 1.

category_mask

Skip pages for which any **PAGE_IS_*** category doesn't match.

category_anyof_mask

Skip pages for which no **PAGE_IS_*** category matches.

return_mask

PAGE_IS_* categories that are to be reported in `page_region`.

RETURN VALUE

On error, -1 is returned, and `errno` is set to indicate the error.

ERRORS

Error codes can be one of, but are not limited to, the following:

EINVAL

Invalid arguments i.e., invalid `size` of the argument, invalid `flags`, invalid `categories`, the `start` address isn't aligned with **PAGE_SIZE**, or `vec_len` is specified when `vec` is NULL.

EFAULT

Invalid `arg` pointer, invalid `vec` pointer, or invalid address range specified by `start` and `end`.

ENOMEM

No memory is available.

EINTR

Fatal signal is pending.

STANDARDS

Linux.

HISTORY

Linux 6.7.

SEE ALSO

ioctl(2)